SkiTrak Assignment 2

Preethi Gajawada | CS6650 - Distributed Systems

Submission package | 10 November 2026 UTC

Git repository

https://github.com/gajawp/SkiTrak

Package contents

Pages 2-3: architecture document (two pages).
Pages 4-8: measured results, graphs, recovery checks and limitations.

Validation

Final builds: server-v2, consumer and client-part2 all succeeded. Server tests: 18; consumer tests: 22; zero test failures or errors. Latest monitored 500,000-swipe run: zero HTTP failures/retries, unique count increased by 500,000, and all ten final leaderboards passed verification.

Known performance limits

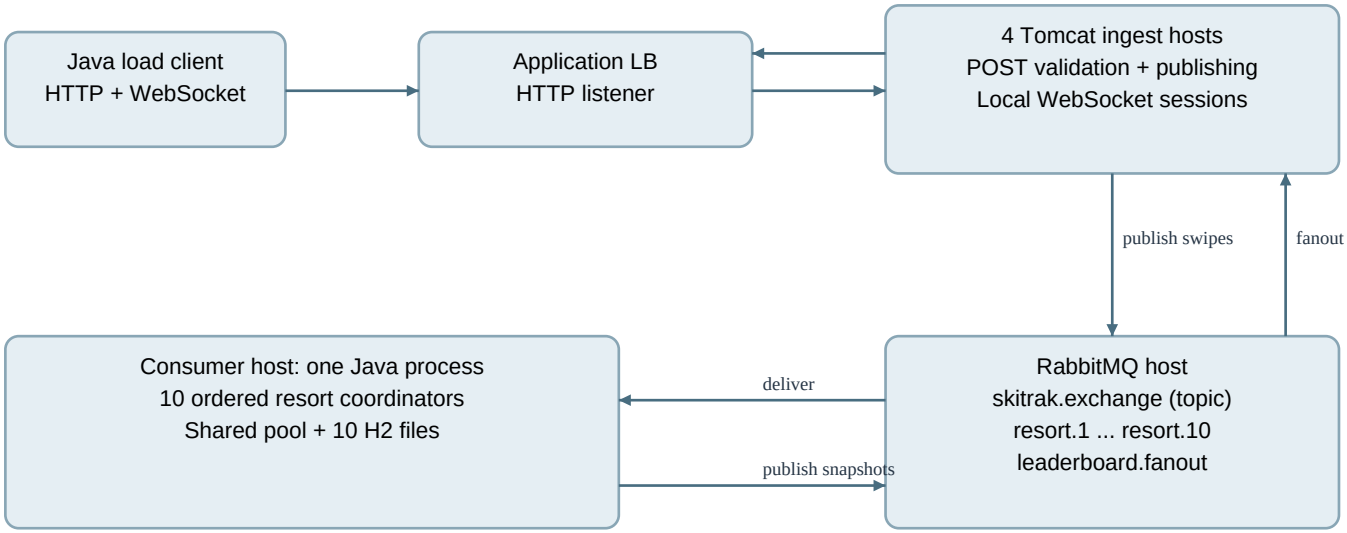
Heavy-load queue depth and consumer-lag targets were not met. The isolated POST-to-WebSocket probes averaged 51.20 ms, but do not represent latency under concurrent load. Single broker and consumer hosts are not a highly available deployment.

Supporting repository material

README.md; AI_USAGE.md; DEPLOYMENT_MONITORING.md; server-v2/, consumer/, deployment/, monitoring/ and selected results/ evidence. The URL identifies the intended submission repository; the final commit and push must be verified before submission.

SkiTrak: Architecture and Reliability

Preethi Gajawada | Deployed lab design | 10 November 2026 UTC



Routing and connection locality

The durable topic exchange skitrak.exchange routes key resort.N to durable queue resort.N, for N=1..10. Queue declarations use a configurable TTL (default 86,400,000 ms), maximum length (default 1,000,000), reject-publish overflow and dead-letter exchange skitrak.dead. Persistent messages and publisher confirms support retry after uncertain HTTP outcomes; finite TTL and capacity remain operational limits.

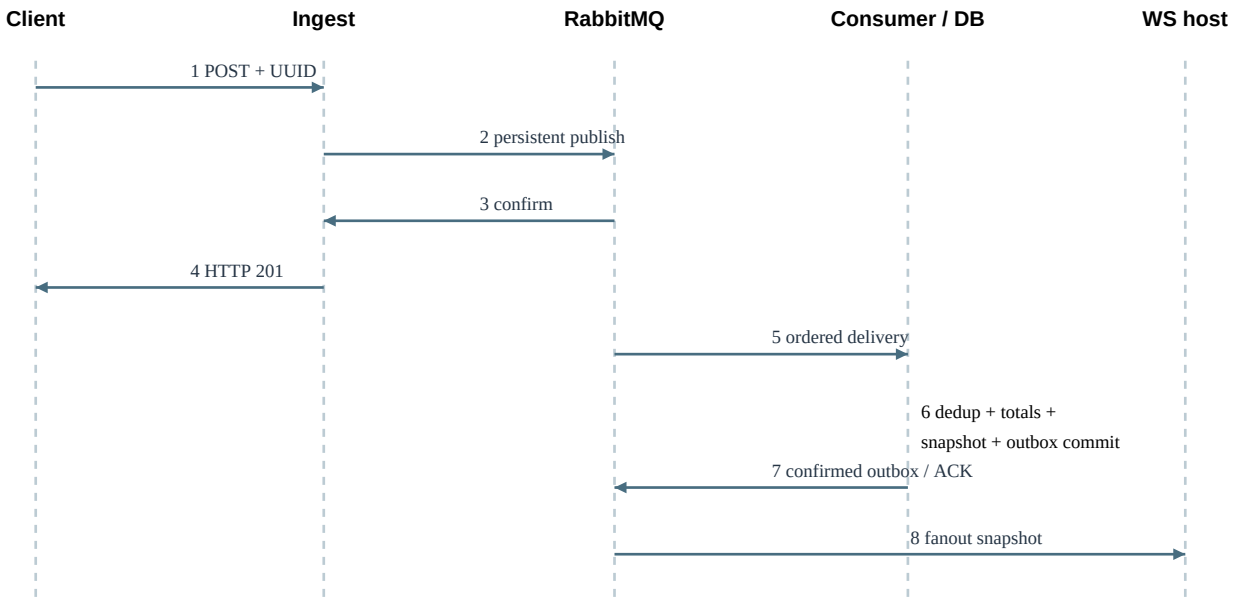
Consumer snapshots publish to durable leaderboard.fanout. Each ingest identity has its own leaderboard.ws.ingest-N queue, so every ingest host receives updates and pushes them to its locally held WebSocket sessions. A retained leaderboard.audit queue also receives updates. This backplane solves connection locality when an HTTP swipe and its subscriber use different ingest hosts. Cookie stickiness cannot by itself move a snapshot to the server holding a socket. Reconnection is required after that server fails.

Ownership, order and aggregation

Each resort has an exclusive queue consumer, one coordinator and one H2 database. A coordinator submits one batch at a time to the shared FIFO executor and waits for completion before reusing its channels. Current batch size/prefetch is 200 with a 20 ms collection wait. Dynamic pool size is 2-10; configured fixed capacities 10/20/40/80 were tested, but active resort tasks cannot exceed ten. HTTP diagnostics use a separate two-thread executor.

Deduplication IDs, skier totals, changed snapshots and an outbox entry commit in one transaction (H2 WRITE_DELAY=0). Same event ID and fingerprint are a duplicate; reuse with different swipe fields is rejected. Top ten is ordered by total vertical descending, then numeric skier ID ascending. Fanout publication is confirmed before input ACKs; replay after failure is safe for aggregation. The design gives idempotent effects with at-least-once delivery, not exactly-once transport.

Swipe to leaderboard sequence



The fanout subscriber sends the snapshot to its local WebSocket sessions. HTTP 201 means broker publication was confirmed; it does not mean aggregation or WebSocket delivery has finished. Snapshot versions change only when the top list changes, so accepted swipes do not imply one WebSocket update each. Subscriber versions and totals are verified independently of transport timestamps.

Load balancer and deployment

One ALB routes HTTP to port 8080 on four t3.micro ingest hosts. Health path /ski-servlet/health uses HTTP 200, interval 30 sec, timeout 5 sec, healthy threshold 2 and unhealthy threshold 3. Routing is round robin; load-balancer cookie stickiness lasts 300 sec. Connection idle timeout was increased from 60 to 300 sec; HTTP client keepalive remains 3,600 sec. Consumer and RabbitMQ each run on a separate t3.small host. AMQP uses the broker private address; diagnostics are accessed through SSH forwarding.

Failure handling and practical limits

Ingest uses one publishing connection per host with bounded channels and sender ownership; HTTP callers wait for their own publish confirmation. Unconfirmed outcomes return 503 and require retry with the same UUID. Resort coordinators reconnect after broker failure. Durable outbox records can be replayed after a crash; a confirm followed by a crash before marking published can duplicate a snapshot. Invalid messages are rejected without requeue. The consumer systemd unit restarts on failure after five seconds; RabbitMQ Docker policy is unless-stopped. Tomcat was manually started after VM restart in these experiments.

Observed recovery: persistent event dedup survived SIGKILL, consumer counts reconciled after an in-flight crash, WebSocket subscribers reconnected after ingest failure, and applications recovered after a graceful broker stop/start. Immediate ALB requests can fail before health detection. The deployment has one broker and one consumer host, and local H2 state: it is not highly available across host or disk loss. Queue depth and consumer lag exceeded targets under heavy load; CPU and I/O measurements support a consumer/storage bottleneck.

SkiTrak Assignment 2

Preethi Gajawada | Results and validation | 10 November 2026 UTC

Measured outcome

The latest four-ingest run accepted and processed all 500,000 swipes without HTTP failures or retries. Final consumer count was 9,810,076, and all ten current leaderboards passed verification. Ingest scaled to thousands of requests per second, but consumer drain time and event-to-commit lag remained substantial.

Latest monitored run	Measured result
Client / resorts	512 main threads; 32 warmup threads; 10 resorts
Workload	500,000 swipes: 32,000 warmup + 468,000 main
Main ingest throughput	7,591.63 swipes/sec
Ingest time / additional drain wait	76.389 sec / 217.480 sec
Wall time / overall throughput	294.214 sec / 1,699.44 swipes/sec
HTTP mean / p95 / p99	63.002 / 130.511 / 214.992 ms
Delivery checks	500,000 successes; zero failures and retries
WebSocket observations	126 updates; 10 connections; zero reconnections or invalid messages

Acceptance limits

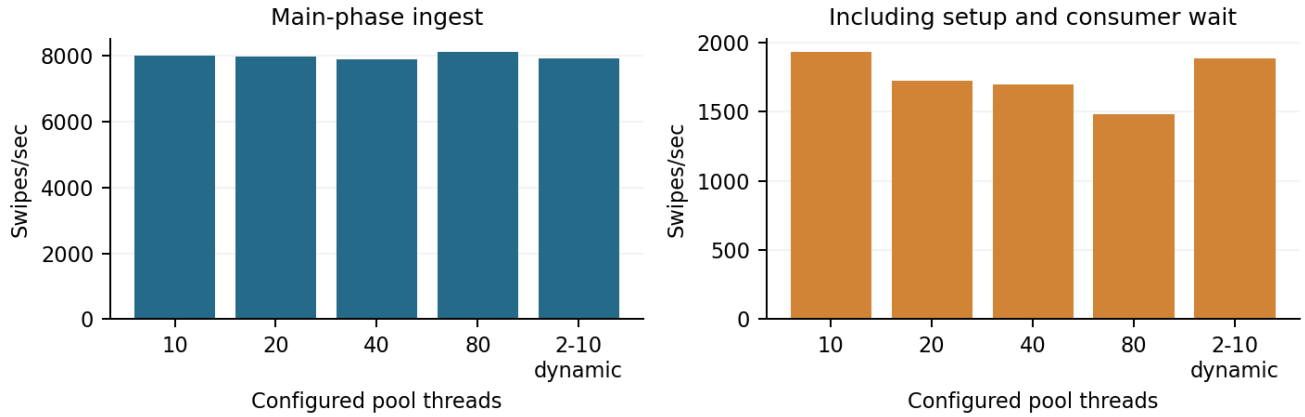
Correctness checks passed; heavy-load latency and queue targets did not. Earlier monitored 500k runs accumulated hundreds of thousands of pending resort messages. The separate 100k latency run had mean event-to-commit lag of 22.425 seconds, with only 0.145% below 100 ms. The successful isolated WebSocket probes must not be used to claim that heavy-load lag is below 100 ms.

Evidence includes user-provided client outputs, RabbitMQ monitoring summaries, six uploaded sar logs and the final verification JSON. This is a results report; repository packaging and the architecture document remain separate deliverables.

Consumer pool comparison

Four ingest servers, 512 main client threads and 500,000 swipes per run. These are sequential experiments, not replicated controlled benchmarks: the persistent database grew between runs and host conditions could change.

Pool	Main ingest /sec	Drain sec	Overall /sec
10	7,990.39	183.644	1,929.68
20	7,973.63	216.669	1,726.03
40	7,897.18	221.095	1,696.14
80	8,097.56	264.783	1,479.41
Dynamic 2-10	7,923.25	191.750	1,884.34



Increasing configured capacity beyond ten did not improve observed completion throughput. The implementation exposes a resort parallelism limit of ten, so capacities of 20, 40 and 80 do not demonstrate that those numbers of aggregation tasks executed simultaneously. Fixed ten had the best completion throughput among these individual runs; repetitions would be needed to establish a reliable ranking.

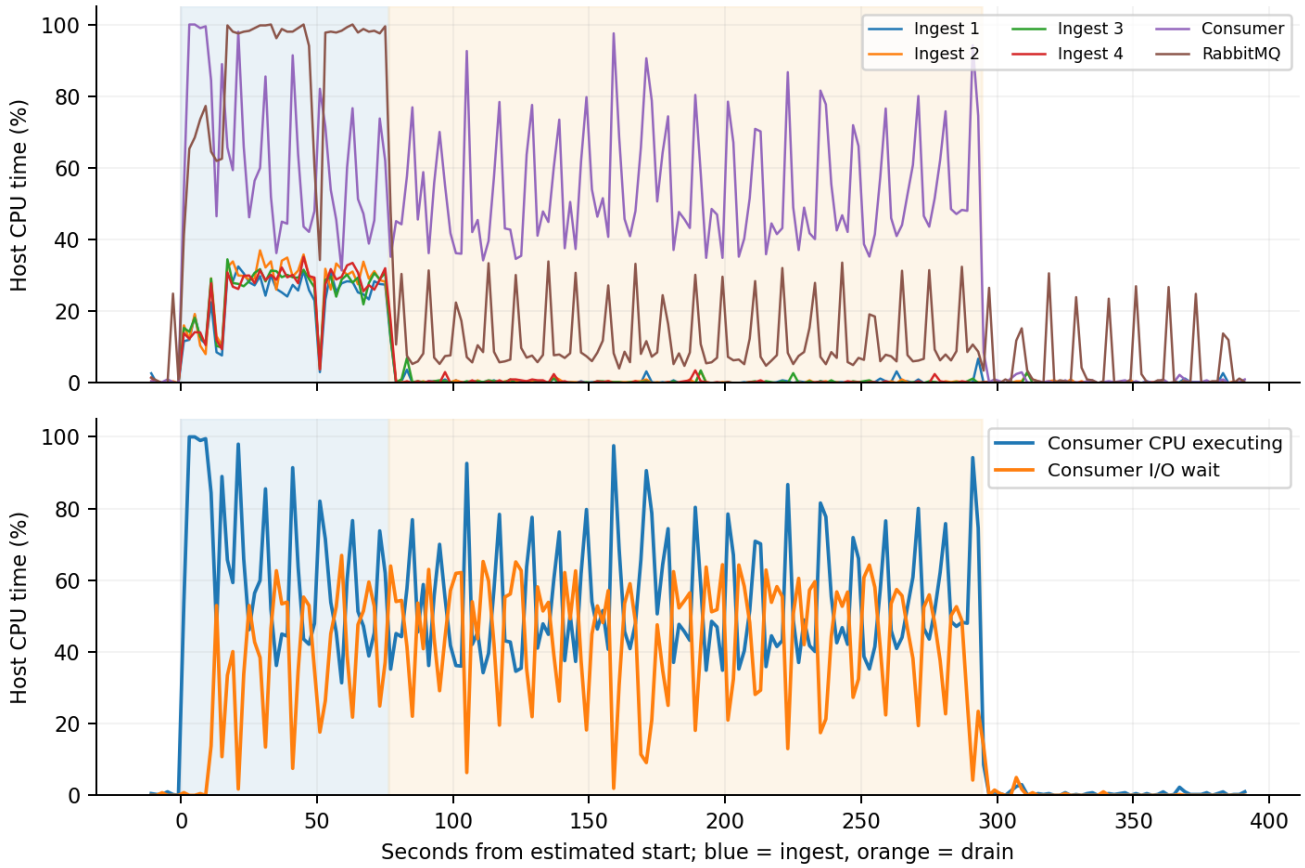
Other workload coverage

A four-ingest 1,000,000-swipe run completed with no HTTP failures: main throughput 7,118.89/sec, ingest time 151.161 sec, additional drain 328.133 sec and wall time 479.791 sec. Historical four-ingest 64- and 128-thread runs recorded main throughput of 3,492.72/sec and 5,521.96/sec. A historical 256-thread run used two ingest servers; it is not a matched four-server comparison.

The latest monitored dynamic run reached 7,591.63/sec main ingest and 1,699.44/sec including setup and drain. Report it separately from the earlier dynamic experiment rather than substituting one result into the other.

CPU and storage pressure

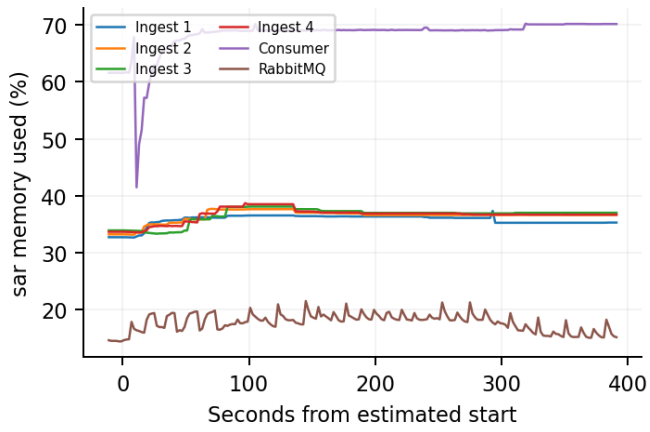
Host-level sar samples were collected every two seconds. Each uploaded log contains 202 samples, from 00:37:02 to 00:43:44 UTC. Recordings cover the workload and drain, but are shorter than the intended ten minutes. Phase boundaries below are approximate: result-directory timestamp 00:42:07.161 UTC minus recorded wall duration estimates a start near 00:37:12.947 UTC.

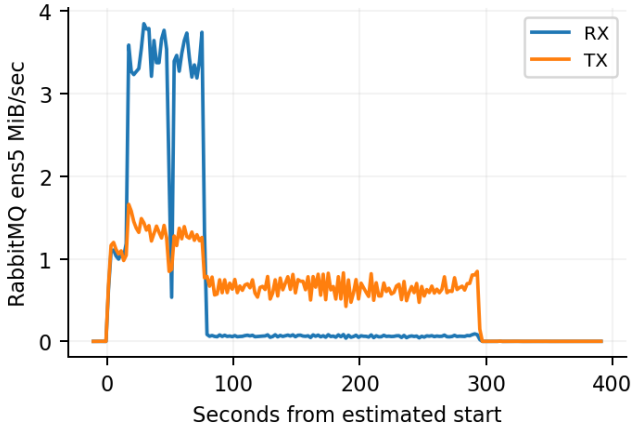


Phase	Consumer CPU executing	Consumer I/O wait
Ingest window	63.6% average	33.7% average
Drain window	54.1% average	44.5% average

CPU executing is %user + %nice + %system across the host, excluding I/O wait and steal. RabbitMQ averaged 88.5% executing CPU during ingest, while the four ingest hosts averaged 23.7-26.9%. During drain the ingest hosts were mostly idle, but consumer CPU and I/O wait remained high. This supports a consumer/storage limitation, with additional broker pressure during publication. sar alone cannot attribute the wait to a particular database operation or disk device.

Memory and network





Host	Peak sar memory used	Peak RX / TX MiB/sec
Ingest 1	37.33%	1.24 / 1.64
Ingest 2	37.69%	1.26 / 1.66
Ingest 3	38.09%	1.25 / 1.65
Ingest 4	38.69%	1.24 / 1.64
Consumer	70.21%	1.04 / 0.12
RabbitMQ	21.50%	3.85 / 1.66

Memory figures are host-level sar values, not Java heap or process RSS. The consumer retained at least 393.88 MiB available across the recording. Network rates use the physical ens5 interface only; Docker bridge and veth traffic are excluded to avoid double counting. Observed traffic alone does not establish the interface capacity or rule out transient network limits.

Commit and WebSocket latency are different measurements

Measurement / scope	Result
100k load: ingest timestamp to successful commit	100,000 samples; mean 22,425.01 ms; maximum 39,378.25 ms; 145 samples below 100 ms
100k load: applyBatch processing	767 successful batches; mean 642.38 ms; max 1,737.33 ms. Excludes queue and pool wait, ACK and publication.
20 isolated POST-to-WebSocket probes	Mean 51.20 ms; median 51.34 ms; nearest-rank p95 52.38 ms; max 52.54 ms

The isolated probes used one event at a time and the Mac monotonic clock, from POST dispatch to matching WebSocket receipt. Versions advanced from 414 to 434 and each count check passed. This includes scheduling, HTTP, ingestion, queue, commit and delivery; it is neither WebSocket-only latency nor concurrent-load latency. Cross-host commit measurements were preceded by synchronized-clock checks.

Recovery and final correctness

Test	Observed evidence and scope
ALB distribution and stickiness	40 requests without cookie reuse reached all four ingest IDs. Ten cookie-preserving requests stayed on ingest-2.
Ingest failure	An immediate request after shutdown returned 502. Once ingest-2 became unhealthy, a cookie-preserving request returned 201 from ingest-3. All four targets were later restored healthy.
WebSocket reconnection	Four observed subscribers reconnected after ingest-2 stopped. Resort 2 received another update after a fresh swipe. Exact outage latency was not established.
Persistent deduplication	Same UUID submitted twice increased unique count once. Following SIGKILL and automatic restart, resubmitting that UUID did not increase the count.
Consumer crash under load	Near-crash queue sample showed 2,000 unacknowledged messages. Client completion polling failed during the outage; later count reconciliation accounted for all 100,000 swipes and leaderboards passed. Exact redelivery count was not measured.
Broker stop/start	Ingest health returned 503 during broker downtime. After restart, all resort and backplane consumers reattached, host health recovered and a new swipe was processed. This was a graceful stop/start, not a pending-message crash-durability test.

Final verification at 2026-11-10 00:48:00 UTC passed all ten resorts and reported a stable consumer count of 9,810,076. Current snapshots agree; this does not retrospectively prove every live WebSocket update was received.

Final process-local metrics cover **600,020 fresh events**: the 100k run, 20 isolated probes and latest 500k run. Their combined mean commit lag was 88,239.56 ms, maximum 218,334.68 ms and 0.3465% below 100 ms. These cumulative values must not be presented as the latest 500k run alone.

Evidence files: leaderboard-verification-20261109-164800.json; six host-IP .log files; client result directory part2-512threads-1794271327161; isolated probe report websocket-latency-20261110-003547-014522.json. Historical scaling and recovery numbers above come from recorded terminal outputs; retain those raw logs with the submission.